

# Fundamentals of Relational Databases

**Presented by:**

Cristian Camilo Cortes Ortiz

# Table of contents

|                                        |           |
|----------------------------------------|-----------|
| <b>Historical context of databases</b> | <b>5</b>  |
| <b>Types of databases</b>              | <b>6</b>  |
| Relational databases (RDB)             | 6         |
| Non-relational databases               | 6         |
| <b>Services</b>                        | <b>7</b>  |
| Self-managed services                  | 7         |
| Managed services                       | 7         |
| <b>Relational databases (RDB)</b>      | <b>7</b>  |
| <b>Relational algebra</b>              | <b>7</b>  |
| <b>Codd's rules</b>                    | <b>8</b>  |
| - Rule 0                               | 8         |
| - Rule 1                               | 8         |
| - Rule 2                               | 8         |
| - Rule 3                               | 8         |
| - Rule 4                               | 8         |
| - Rule 5                               | 8         |
| - Rule 6                               | 8         |
| - Rule 7                               | 8         |
| - Rule 8                               | 8         |
| - Rule 9                               | 8         |
| - Rule 10                              | 8         |
| - Rule 11                              | 8         |
| - Rule 12                              | 8         |
| <b>Attributes</b>                      | <b>9</b>  |
| <b>Database Normalization</b>          | <b>10</b> |
| <b>Strong Entity</b>                   | <b>11</b> |
| <b>Weak Entity</b>                     | <b>12</b> |
| <b>Relationships</b>                   | <b>13</b> |
| <b>Cardinality</b>                     | <b>13</b> |
| - Cardinality: 1 to 1                  | 13        |
| - Cardinality: 0 to 1                  | 13        |
| - Cardinality: 1 to N                  | 13        |
| - Cardinality: 0 to N                  | 13        |
| - Cardinality: N to N                  | 13        |
| <b>Relational model</b>                | <b>14</b> |
| <b>Entity relationship model</b>       | <b>15</b> |
| <b>Data types</b>                      | <b>16</b> |
| <b>RDMS</b>                            | <b>17</b> |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>MySQL Workbench</b>                        | <b>18</b> |
| <b>PgAdmin</b>                                | <b>19</b> |
| <b>SQL - NoSQL</b>                            | <b>19</b> |
| <b>SQL commands</b>                           | <b>20</b> |
| DDL                                           | 21        |
| - CREATE                                      | 21        |
| - ALTER                                       | 21        |
| - DROP                                        | 21        |
| - TRUNCATE                                    | 21        |
| - COMMENT                                     | 21        |
| - RENAME                                      | 21        |
| Database                                      | 22        |
| Table                                         | 22        |
| View                                          | 22        |
| DML                                           | 23        |
| - SELECT                                      | 23        |
| - INSERT                                      | 23        |
| - UPDATE                                      | 23        |
| - DELETE                                      | 23        |
| - MERGE                                       | 23        |
| - CALL                                        | 23        |
| - EXPLAIN PLAN                                | 23        |
| - LOCK TABLE                                  | 23        |
| - CASCADE                                     | 23        |
| - SET NULL                                    | 23        |
| - RESTRICT                                    | 23        |
| - NO ACTION                                   | 24        |
| DCL                                           | 24        |
| - GRANT                                       | 24        |
| - REVOKE                                      | 24        |
| TCL                                           | 24        |
| - COMMIT                                      | 24        |
| - ROLLBACK                                    | 24        |
| - SAVEPOINT                                   | 24        |
| - SET TRANSACTION                             | 24        |
| <b>STORAGE ENGINES</b>                        | <b>25</b> |
| The main advantages of InnoDB over MyISAM are | 26        |
| The main advantages of MyISAM over InnoDB are | 26        |
| <b>Queries</b>                                | <b>26</b> |
| <b>Command execution order</b>                | <b>27</b> |
| <b>Anatomy of a query</b>                     | <b>27</b> |

|                                           |           |
|-------------------------------------------|-----------|
| <b>Nested queries</b>                     | <b>28</b> |
| <b>From question to query</b>             | <b>29</b> |
| - SELECT                                  | 29        |
| - FROM                                    | 29        |
| - WHERE                                   | 29        |
| - DELETE                                  | 29        |
| - LIKE                                    | 29        |
| - BETWEEN                                 | 29        |
| - GROUP BY                                | 29        |
| - ORDER BY                                | 29        |
| - HAVING                                  | 29        |
| - LIMIT                                   | 29        |
| - OFFSET                                  | 29        |
| - FETCH                                   | 29        |
| <b>Joins</b>                              | <b>30</b> |
| Relational Recombination                  | 30        |
| Primary/Foreign Key Alignment             | 30        |
| Join Types and Match Logic                | 30        |
| Query Performance                         | 30        |
| <b>Views</b>                              | <b>32</b> |
| Virtual Nature                            | 32        |
| Query Simplification                      | 32        |
| Security and Data Abstraction             | 32        |
| Centralized Business Logic                | 32        |
| <b>Stored Procedures</b>                  | <b>33</b> |
| Procedural Logic                          | 33        |
| Performance Efficiency                    | 33        |
| Transaction Control                       | 33        |
| Security and Encapsulation                | 33        |
| <b>Triggers</b>                           | <b>34</b> |
| Event-Driven Execution                    | 34        |
| Timing and Granularity                    | 34        |
| Special Pseudo-Records                    | 34        |
| Automated Auditing and Advanced Integrity | 34        |
| <b>Non-relational databases</b>           | <b>35</b> |
| <b>Types of non-relational databases</b>  | <b>35</b> |
| Key-Value                                 | 35        |
| Document-based                            | 36        |
| Graph-based                               | 36        |
| In-memory                                 | 37        |
| Search-optimized                          | 37        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Managed services and data hierarchy</b>              | <b>38</b> |
| Firebase                                                | 38        |
| Firestore                                               | 38        |
| Data types in Firestore                                 | 40        |
| String                                                  | 40        |
| Number                                                  | 40        |
| Boolean                                                 | 40        |
| Map                                                     | 40        |
| Array                                                   | 40        |
| Null                                                    | 40        |
| Timestamp                                               | 40        |
| Geopoint                                                | 40        |
| Reference                                               | 40        |
| <b>Handling data models in non-relational databases</b> | <b>41</b> |
| Top-level collections                                   | 41        |
| Documents                                               | 41        |
| Subcollections                                          | 41        |
| <b>Databases in real life</b>                           | <b>42</b> |
| <b>Big data</b>                                         | <b>42</b> |
| <b>Data warehouse</b>                                   | <b>44</b> |
| <b>Data mining</b>                                      | <b>46</b> |
| <b>ETL</b>                                              | <b>48</b> |
| <b>Business Intelligence</b>                            | <b>49</b> |
| <b>Machine Learning</b>                                 | <b>51</b> |
| Classification                                          | 52        |
| Prediction                                              | 52        |
| <b>Data science</b>                                     | <b>53</b> |

## Historical context of databases

With the digital media revolution of the 1920s, information began to be stored on magnetic tape and hard drives, which used binary bits and bytes.

The evolution of computing toward the cloud allows access to information stored on hard drives from anywhere in the world. It is a centralized medium that allows multiple people to use it simultaneously.

The concept of databases originated during the period when the shift to digital media—and now to the cloud took place. Databases complement the von Neumann architecture, the classic computing model (CPU, RAM, input and output devices).

## Types of databases

Databases are divided into two groups:

- Relational
- Non-relational

Historically, relational databases emerged first, followed by non-relational databases.

### Relational databases (RDB)

- Microsoft SQL Server
- Oracle
- MariaDB
- PostgreSQL
- MySQL

### Non-relational databases

- cassandra
- elastic search
- DynamoDB
- BigQuery
- neo4j
- MongoDB
- Firestore

## Services

- Self-managed
- Managed

### Self-managed services

These are databases installed on a computer or server; the administrator is responsible for updates, maintenance, data consistency, and related tasks.

### Managed services

These are database services offered by cloud computing companies such as AWS, Google Cloud, and Azure. The administrator is responsible for using the service, but not for installation, maintenance, or support.

## Relational databases (RDB)

The Von Neumann architecture model includes main memory (RAM) and data processing, but it does not address persistent data storage; that concept emerged later. Driven by the need to store information in a portable manner, the concept of file-based databases was developed, which involves storing information in plain text using formats. One of them is CSV (Comma-Separated Values), a format that makes interacting with the information more complex. This concept gave rise to the need for more structured information, leading to the development of relational databases (RDB).

## Relational algebra

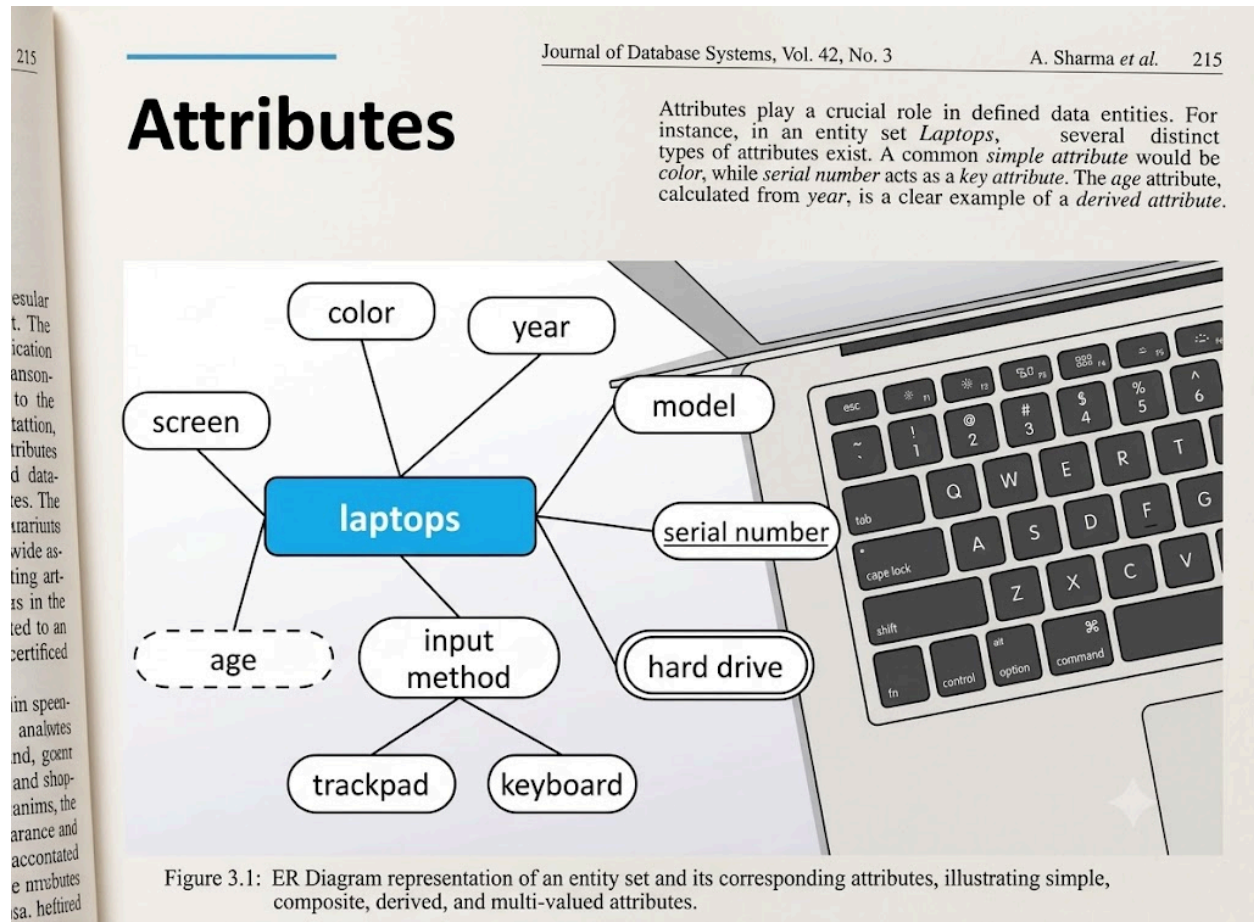
Relational algebra is more closely related to data structures, which allows data to be combined and joined based on various properties and characteristics.

## Codd's rules

Edgar Codd is the inventor of relational databases and relational algebra. Codd's 12 rules are a set of 13 rules (numbered 0 through 12) proposed by Edgar Codd for the relational model of databases, designed to define the requirements for a database management system.

- **Rule 0**  
Foundational Rule
- **Rule 1**  
Information Rule
- **Rule 2**  
Guaranteed Access Rule
- **Rule 3**  
Rule for the Systematic Handling of Null Values
- **Rule 4**  
Dynamic Online Catalog Based on the Relational Model
- **Rule 5**  
Rule Encompassing the Data Sublanguage
- **Rule 6**  
Rule for Updating Views
- **Rule 7**  
High Level of Insertion, Update, and Deletion
- **Rule 8**  
Physical Independence of Data
- **Rule 9**  
Logical Independence of Data
- **Rule 10**  
Independence of Integrity
- **Rule 11**  
Distribution Independence
- **Rule 12**  
Non-Subversion Rule

## Attributes



| Serial number   | color         | year | model        |
|-----------------|---------------|------|--------------|
| WBACW788423312  | Alpine White  | 2019 | 330i G20     |
| WBACW724312323  | Estoril Blue  | 2023 | X5 xDrive40i |
| WBRT24324324376 | Mineral Grey  | 2022 | M4 G82       |
| WMKT7655467433  | Grey Metallic | 2021 | 530i G60     |

# Database Normalization

It is the systematic process of organizing data in a relational database to reduce redundancy and improve data integrity. It progresses through several numbered stages known as Normal Forms:

## First Normal Form (1NF)

Eliminates repeating groups and ensures that each field contains only atomic (indivisible) values.

## Second Normal Form (2NF)

Builds on 1NF by removing partial dependencies, ensuring that all non-key attributes are fully dependent on the entire primary key.

## Third Normal Form (3NF)

Extends 2NF by removing transitive dependencies, meaning non-key attributes must depend solely on the primary key and not on other non-key attributes.

## Higher Normal Forms

Beyond 3NF, databases can be further normalized using **Boyce-Codd Normal Form (BCNF)**, a stricter variant of 3NF that handles complex composite keys, followed by **Fourth Normal Form (4NF)** to address multivalued dependencies, and **Fifth Normal Form (5NF)** to handle join dependencies without data loss.

## Strong Entity

An entity is the equivalent of an object in object-oriented programming languages.

In a flowchart, objects are represented within rectangles.

Attributes are represented within ovals. When an object has multiple elements with the same attribute, that attribute is represented within double ovals.

Composite attributes are those composed of other attributes; special attributes are represented within ovals with dotted lines.

By convention, entities are written in the plural because they represent groups of objects.

A key attribute helps prevent data redundancy. Natural key attributes are inherent to the object; if the object does not have a key attribute, an artificial key attribute is added. Since it is not inherent to the object itself, it is assigned arbitrarily.

## Weak Entity

Weak entities depend on another entity for their existence.

In a flowchart, objects are represented within rectangles with an internal line.

Weak entities can be weak for two reasons: identity or existence.

Weak entities by identity differ from one another only by their strong identity key.

When a weak entity has its own unique identifier, it ceases to be weak by identity and becomes weak by existence because the object has its own identifier; however, conceptually, an identity cannot exist without the entity itself existing. Although it does not depend on the strong entity's identifier, it is tied to the strong entity's existence.

When identifying the attributes of a user entity, using the email address as an ID is not a best practice because the user may change it at some point, which would make it difficult to link content associated with the old email address; therefore, adding an artificial key facilitates the creation of the user entity.

A best practice is to start by creating entities that do not have a foreign key.

## Relationships

Relationships are the way in which different entities or objects are linked together.

In a flowchart, relationships are represented by a diamond.

When multivalued attributes exist, they are generally treated as separate entities because they have a life of their own and because they can be related in different ways.

Transitive tables serve as a bridge to link multiple tables. They have no semantic content.

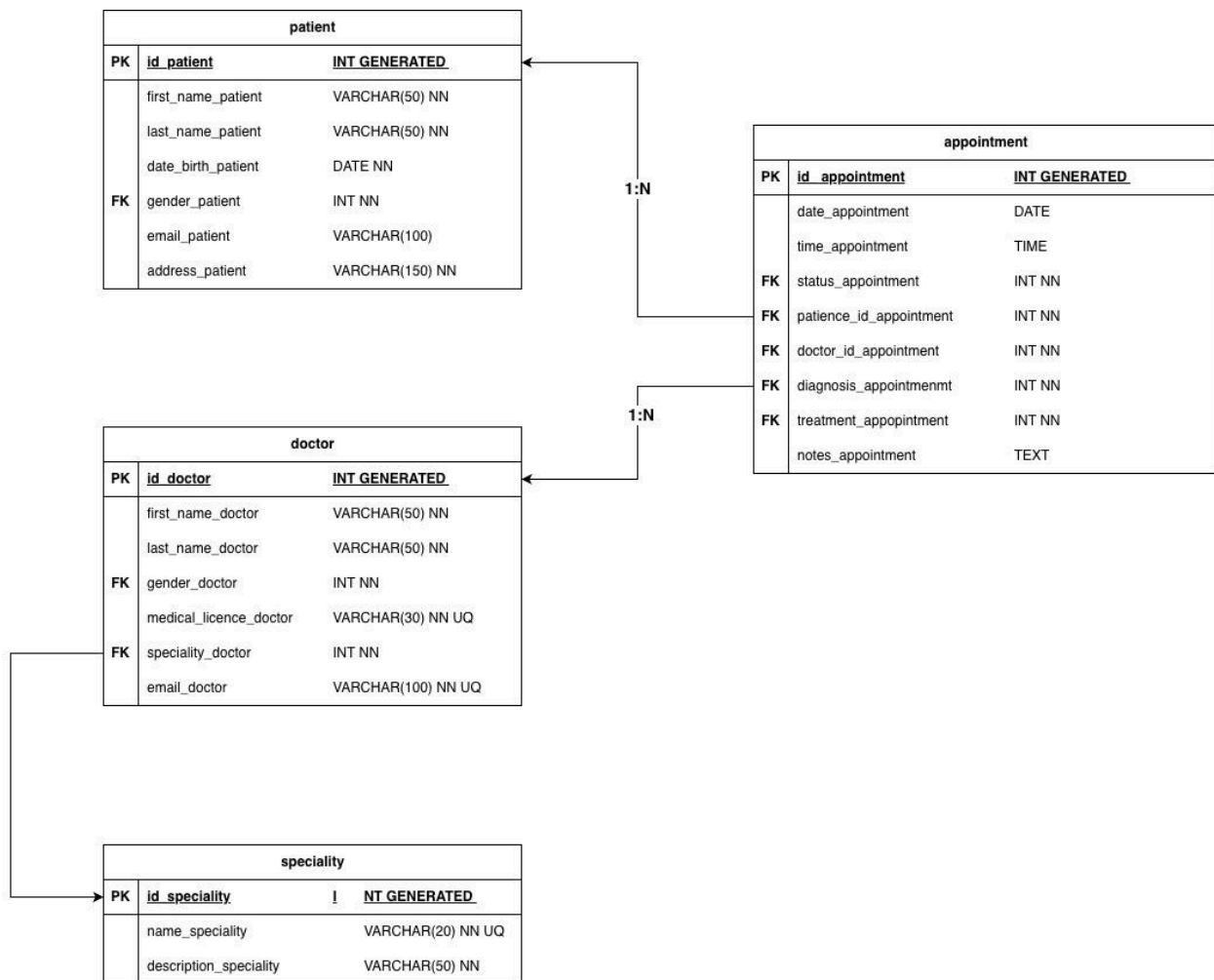
## Cardinality

Cardinality is a property of relations that relates to numbers; its purpose is to identify how many elements on one side belong to the other side. There are different types of cardinality.

- **Cardinality: 1 to 1**  
Is defined by the statement, taking the larger number of the two sides.
- **Cardinality: 0 to 1**  
Is defined because some of the sides may or may not exist.
- **Cardinality: 1 to N**  
Is defined such that one side has 1 and the other side may have many.
- **Cardinality: 0 to N**  
Is defined because some of the sides may or may not exist.
- **Cardinality: N to N**  
Is defined because both sides can have many.

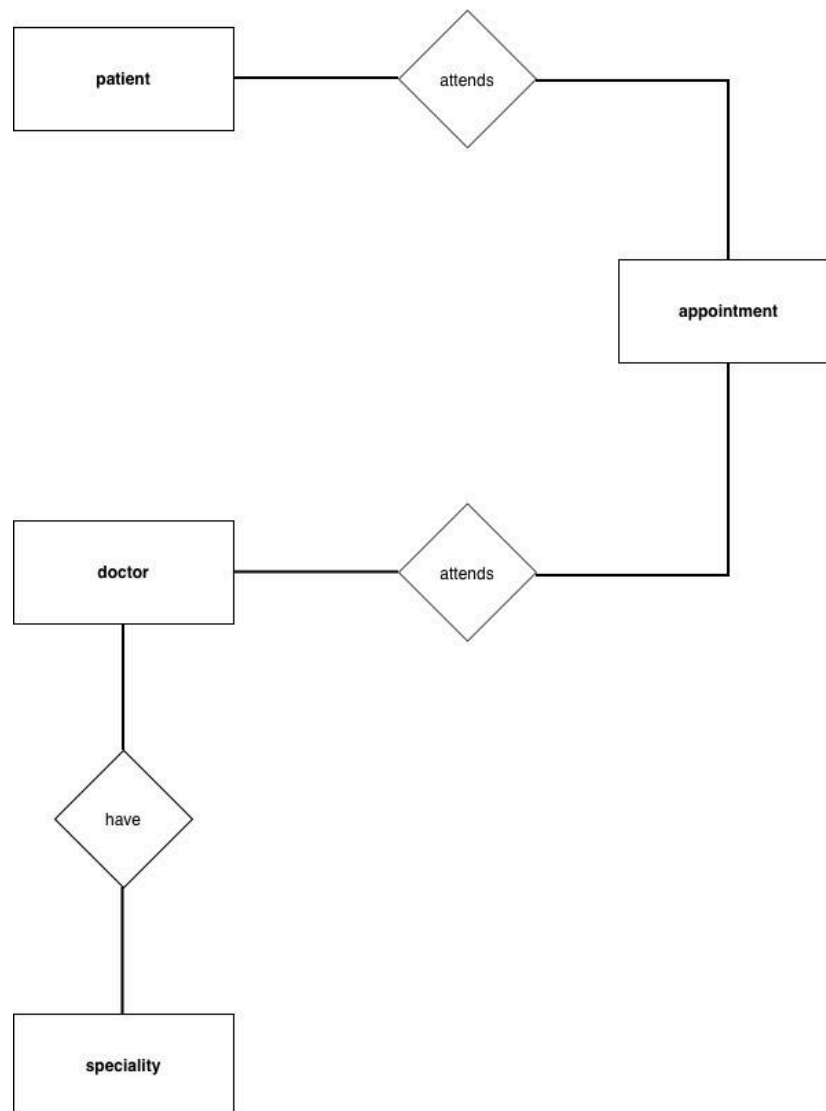
## Relational model

The Relational Model is a foundational approach to database management based on first-order predicate logic, introduced by E.F. Codd in 1970. In this model, data is organized into two-dimensional tables called relations, where each row represents a unique record (or tuple) and each column represents a named attribute (or field). Relationships between entities are established dynamically using shared data values, specifically through primary and foreign keys, rather than physical storage pointers. By enforcing strict constraints to maintain entity and referential integrity, the relational model ensures data consistency while allowing users to manipulate and query information declaratively using languages like SQL without needing to know how the data is physically stored on disk.



## Entity relationship model

The Entity-Relationship (ER) Model is a high-level conceptual data model used to define and structure the data requirements of a database system. Introduced by Peter Chen in 1976, it represents real-world object categories as entities (such as Customer or Order) and the logical connections between them as relationships (such as Places or Belongs To). Entities are defined by their attributes, which describe their properties or characteristics (like Customer\_ID, Name, or Email). By visually organizing these elements into an Entity-Relationship Diagram (ERD) using standardized shapes and cardinality rules (such as 1:1, 1:N, or N:N), database designers can clearly map out complex business domain structures before translating them into physical relational database schemas.



## Data types

| Text       | Numeric      | Date/Time | Logics  |
|------------|--------------|-----------|---------|
| CHAR(n)    | INTEGER      | DATE      | BOOLEAN |
| VARCHAR(n) | BIGINT       | TIME      |         |
| TEXT       | SMALLINT     | DATETIME  |         |
| TINYTEXT   | DECIMAL(n,s) | TIMESTAMP |         |
|            | NUMERIC(n,s) |           |         |

## Constraints

| Constraint  | Description                                                      |
|-------------|------------------------------------------------------------------|
| NOT NULL    | Ensures that the column does not contain null values             |
| UNIQUE      | Ensures that each value in the column is unique                  |
| PRIMARY KEY | A combination of NOT NULL and UNIQUE                             |
| FOREIGN KEY | Uniquely identifies a tuple in another table                     |
| CHECK       | Ensures that the value in the column meets a specified condition |
| DEFAULT     | Sets a default value when no value is specified                  |
| INDEX       | Created per column to allow faster searches                      |

## RDBMS

An RDBMS (Relational Database Management System) is a database management system that enforces Codd's rules. It serves as an interface between databases and the user (providing a graphical interface for managing and interacting with them).

- Oracle
- MySQL
- Postgrest

They all use a base language, but each one adapts it, applying different rules and features.

When installing a MySQL RDBMS on Windows, it is recommended to choose the custom installation and select MySQL Server and MySQL Workbench, based on the computer's architecture.

When configuring a MySQL RDBMS, you should select the configuration type based on the intended use. Be sure to note the port on which the RDBMS will be installed, since if you plan to connect an application or the visual client, you'll need to enter the port to which the RDBMS is connected and set a password.

When installing a MySQL RDBMS on macOS, you must download MySQL Community Server and MySQL Workbench. You must start the MySQL instance in System Preferences so that MySQL Workbench can connect to it.

## MySQL Workbench

An RDBMS (Relational Database Management System) is a database management system that enforces Codd's rules. It serves as an interface between databases and the user (providing a graphical interface for managing and interacting with them).

| Key | Value          |
|-----|----------------|
| PK  | PRIMARY KEY    |
| NN  | NOT NULL       |
| BIN | BINARY         |
| UQ  | UNIQUE         |
| UN  | UNSIGNED       |
| ZF  | ZERO FILL      |
| AI  | AUTO INCREMENT |
| G   | GENERATED      |

Reverse engineering a database in MySQL Workbench generates an EER diagram that provides a better logical understanding of how the database architecture is designed, structured, and implemented.

## PgAdmin

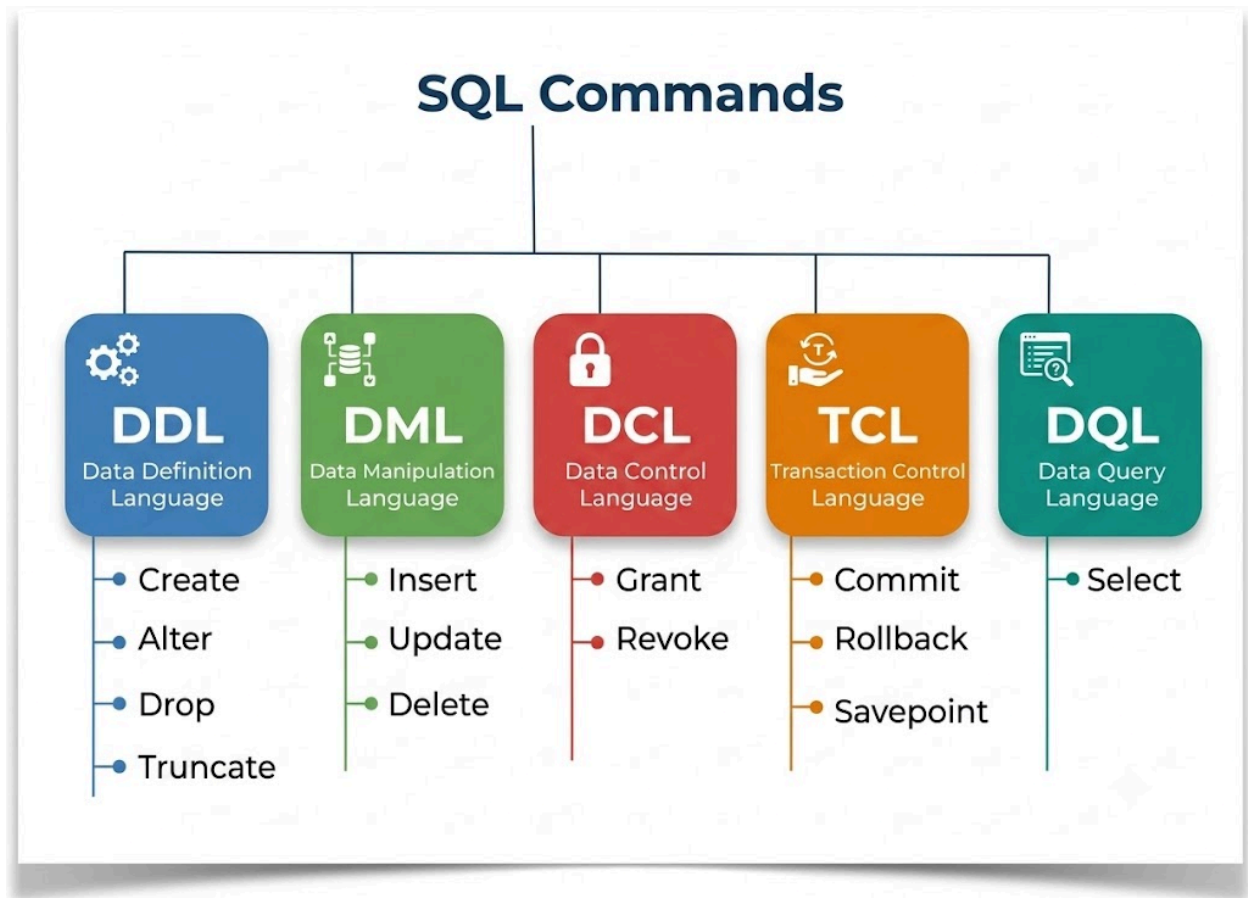
pgAdmin is the leading open-source administration and management platform for PostgreSQL, the world's most advanced open-source relational database. Designed to meet the needs of both novice users and experienced database administrators, pgAdmin offers an intuitive graphical user interface (GUI) that simplifies database operations. It allows users to easily write and debug complex SQL queries, design schema objects, manage user privileges, monitor server performance, and perform automated backups or restores. Available as both a desktop application and a web-based service, pgAdmin serves as an essential, comprehensive tool for streamlining PostgreSQL development and database administration.

## SQL - NoSQL

SQL (Structured Query Language) is a declarative, cross-platform language in the family of query languages. Donald D. Chamberlin and Raymond F. Boyce designed it. The language was developed during a pivotal moment in the history of ICT (Information and Communication Technologies), when people were trying to access data but there was no standard method for doing so. In response to this need, SQL was developed to allow data queries regardless of which database management system is used, as it aims to unify them all, thereby becoming a standard.

NoSQL (Not Only Structured Query Language) refers to languages that allow access to non-relational databases; although NoSQL is not fundamentally based on SQL, it still incorporates SQL.

## SQL commands



## DDL

DDL (Data Definition Language) is a sublanguage that allows you to create the structure of a database (tables, relationships, entities, etc.).

Main commands:

- **CREATE**  
It allows you to create a database or schema, tables, views, indexes, etc.
- **ALTER**  
It allows you to switch between or modify entities, tables, data types, etc.
- **DROP**  
It allows you to delete columns, tables, databases, or schemas, etc.
- **TRUNCATE**  
This deletes all records from a table, including all space allocated to the records. They will be deleted.
- **COMMENT**  
It allows you to add comments to the data dictionary.
- **RENAME**  
It allows you to rename an object.

There are three groups of objects that are manipulated using the DDL sublanguage

- Database
- table
- View

**Database**

It is the infrastructure where data is stored and organized so that it can be accessed.

**Table**

It is the projection or translation into SQL of the relational entities and the physical diagram.

**View**

It is the presentation of data in an understandable way.

## DML

DML (Data Manipulation Language) is a sublanguage that allows you to manipulate data and includes the most common SQL statements (SELECT, INSERT, DELETE, etc.). It is used to store, modify, retrieve, delete, and update data in the database

Main commands:

- **SELECT**  
It allows you to retrieve information from the database.
- **INSERT**  
It allows you to insert data into a table in the database.
- **UPDATE**  
It allows you to update existing data in a table in the database.
- **DELETE**  
This allows you to delete all records from a table in the database.
- **MERGE**  
It supports the UPSERT operation for inserting or updating.
- **CALL**  
Allows you to call a PL/SQL or Java subprogram.
- **EXPLAIN PLAN**  
It allows you to interpret the path to the data.
- **LOCK TABLE**  
It enables concurrency control.
- **CASCADE**  
Ensures that every time an UPDATE is performed on the main table, it is also reflected in the table with which the relationship is being established.
- **SET NULL**  
Set it so that every time an UPDATE is performed on the main table, the data is set to NULL.
- **RESTRICT**  
Ensures that whenever an UPDATE is performed on the main table, the change is not applied until it is resolved first.

- **NO ACTION**

Ensures that whenever an UPDATE is performed on the main table, no action is taken on the table with which the relationship is being established.

## DCL

DCL (Data Control Language) is a sublanguage that allows you to control data; it is related to rights, permissions, and other controls within the database system.

Main commands:

- **GRANT**

It allows you to assign database access privileges to users.

- **REVOKE**

Allows you to revoke access privileges granted to users using the GRANT command.

## TCL

TCL (Transaction Control Language) is a sublanguage that allows you to manage transactions within a database.

Main commands:

- **COMMIT**

Allows you to commit or approve a transaction.

- **ROLLBACK**

It allows you to reverse a transaction in the event of an error.

- **SAVEPOINT**

It allows you to roll back a transaction by creating checkpoints within groups.

- **SET TRANSACTION**

Allows you to specify the transaction details.

## STORAGE ENGINES

The storage engine is responsible for storing, managing, and retrieving information from a table. The most well-known storage engines are MyISAM and InnoDB. The choice between them depends on the specific use case.

In cases where transactions, foreign keys, and locks are required, InnoDB should be chosen.

In cases where SELECT queries to the database predominate, MyISAM should be chosen.

InnoDB provides MySQL with a transactional storage engine (ACID-compliant) (Atomicity, Consistency, Isolation, and Durability) with commit, rollback, and fault recovery capabilities. InnoDB performs row-level locking but also provides Oracle-style consistent read-without-locking capabilities in SELECT statements. These features enhance performance and the ability to handle multiple concurrent users. Scaled locking is not required in InnoDB because row-level locks take up very little space; InnoDB also supports FOREIGN KEY constraints. In SQL queries, even within the same query, InnoDB tables can be freely joined with tables of other types.

MyISAM is the default engine. To create an InnoDB table, you must specify the option ENGINE = InnoDB or TYPE = InnoDB in the SQL statement used to create the table.

## The main advantages of InnoDB over MyISAM are

- Transaction Support
- Record Locking
- Provides ACID properties, ensuring table integrity
- If the application increases its use of INSERT and UPDATE statements, you will likely see improved performance compared to MyISAM

## The main advantages of MyISAM over InnoDB are

- Faster overall data retrieval
- Recommended for applications where SELECT statements predominate over INSERT and UPDATE statements
- Lacks atomicity features, since it does not have to perform referential integrity checks or lock tables to carry out operations; this results in faster performance.

## Queries

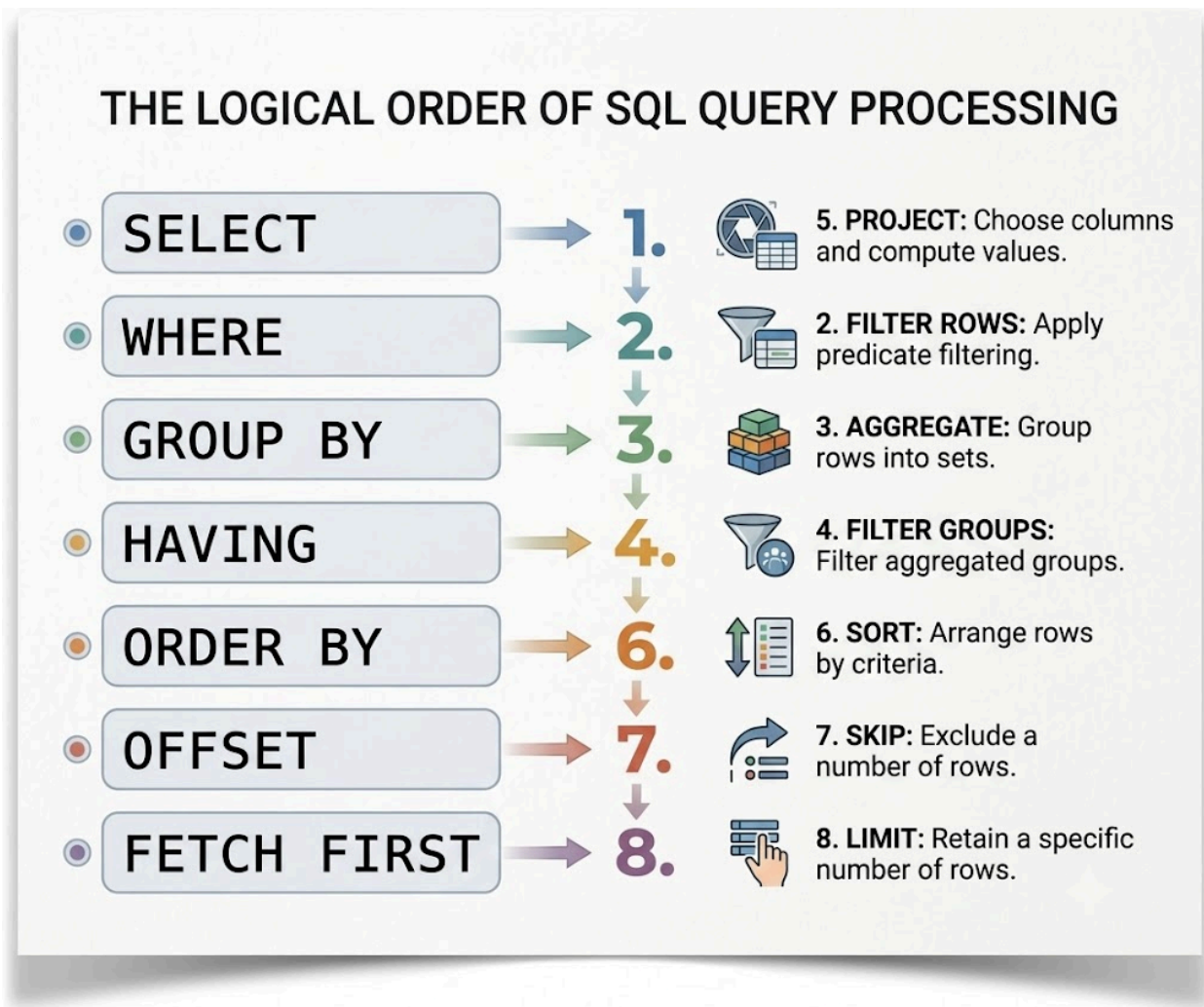
Database queries are essential, as they provide information that could save a business or company.

Several specializations have emerged around database queries, such as ETL (Extract, Transform, and Load), BI (business intelligence), DS (data science), and even ML (machine learning).

Queries are the way in which the questions to be asked of the database are structured.

The basic structure of a query consists of the SELECT statement, the FROM statement, and the WHERE statement. The minimum required statements are SELECT and FROM.

## Command execution order



## Anatomy of a query

```

UPDATE clause { UPDATE country
                SET clause { SET population = population + 1
                WHERE clause { WHERE name = 'USA' ;
                                expression
                                predicate
                                }
                                }
                                } statement
  
```

The diagram shows the anatomy of a query. It consists of three clauses: UPDATE, SET, and WHERE. The UPDATE clause is followed by a SET clause, which is followed by a WHERE clause. The SET clause contains the expression **population + 1**. The WHERE clause contains the predicate **name = 'USA'**. The entire query is enclosed in a large curly brace on the right, labeled **statement**.

## Nested queries

Nested queries, or subqueries, are queries nested within one another (a query inside another query) and are very useful in specific cases, such as creating a table that performs a JOIN with another table a table that exists only in memory and is also used to perform a JOIN with an existing table in the database or as the final result of a query in a WHERE clause or condition.

Nested queries are generally used when there is a specific problem that cannot be solved simply by querying the table in the database, and it is necessary to add or perform another type of operation involving data from another table or from the same table; in such cases, when grouping the data, it is not possible to use a WHERE clause with an additional condition.

As the data set grows, say, to 1,000 or 1,000,000 records to query when queries become nested, a Cartesian product is created, which involves multiplying every record in one table by every record in the new query, once for each record. This nesting causes the query to become too heavy and difficult for the RDBMS to process (depending on the server and its computing capacity); generally speaking, it does not scale and results in exponential complexity. For every query executed within the super-query, a corresponding number of queries will be executed within the sub-query; thus, every time a record is added to the super-query, the number of queries in the sub-query increases by that same factor, causing the total to multiply and grow exponentially.

Nested queries are useful in specific situations where the query itself is not scalable; that is, when the table on which the query is performed does not grow constantly.

## From question to query

When formulating questions to be used in database queries, it is important to keep in mind that the statements in a query are equivalent.

- **SELECT**  
What we want to show.
- **FROM**  
Where will the data come from?
- **WHERE**  
Filters for the data you want to display.
- **DELETE**  
This allows you to delete all records from a table in the database.
- **LIKE**  
Filters records when only part of the information is known.
- **BETWEEN**  
Filters the records within a specific range.
- **GROUP BY**  
The categories by which we want to organize the information.
- **ORDER BY**  
The order in which you want the information to be displayed.
- **HAVING**  
The filters for the grouped data you want to display.
- **LIMIT**  
The specified number of records to be displayed.
- **OFFSET**  
The specified number of the first records to skip.
- **FETCH**  
The specific total number of records to be displayed.

## Joins

As databases grow, information is often distributed across multiple related tables instead of being stored in a single table. SQL JOINS allow us to combine data from these tables based on relationships between their columns. For example, in an airline reservation system, passenger information may be stored in one table while reservations, flights, and airports are stored in others. Using JOINS, we can retrieve related information from these tables in a single query. Understanding JOINS is essential for working with relational databases because they allow us to connect related records, answer more complex questions, and obtain meaningful information from normalized data.

### Relational Recombination

Blends normalized data spread across multiple tables back into a single, comprehensive result set.

### Primary/Foreign Key Alignment

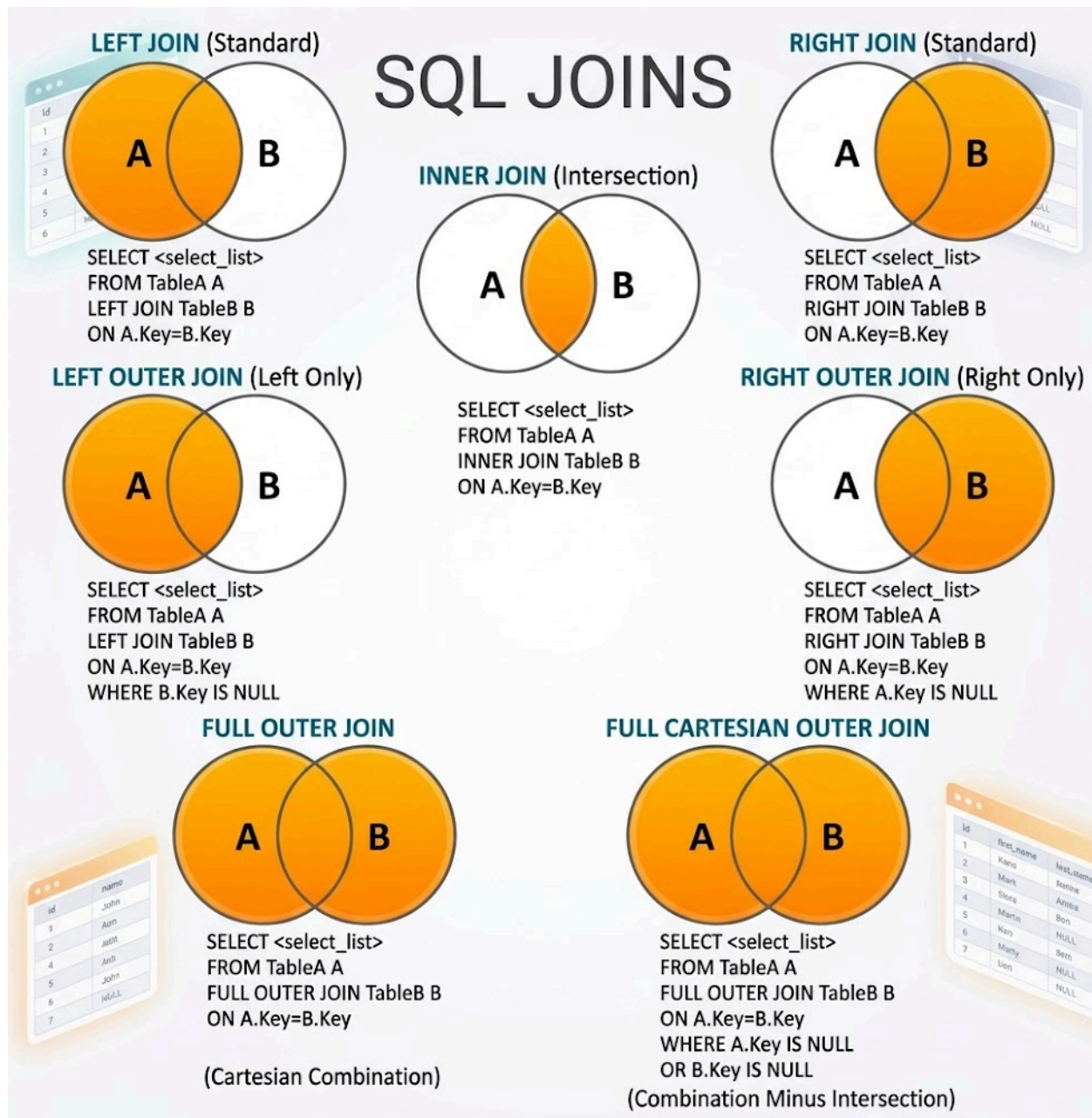
Operates by matching related attributes, usually linking a primary key in one table to a foreign key in another.

### Join Types and Match Logic

Determines row inclusion: INNER JOIN requires matches on both sides, while LEFT, RIGHT, and FULL outer joins retain unmatched rows by padding missing values with NULL.

### Query Performance

Requires strategic indexing on foreign key columns to avoid expensive table scans and unintended Cartesian products (CROSS JOIN).



# Views

In relational database management systems, a View is a virtual table whose content is defined by a stored `SELECT` query. Unlike base tables, standard views do not physically store data on disk; instead, they dynamically retrieve and display data from one or more underlying tables when queried. Views act as a powerful abstraction layer in database architecture. They allow developers to simplify complex queries such as multi-table joins, subqueries, and aggregations into clean, reusable objects. Furthermore, views enhance security and data governance by allowing administrators to restrict user access to specific rows or columns without granting direct permissions to sensitive base tables. By encapsulating query logic inside a view, database systems achieve greater modularity, maintainability, and consistency across applications.

## Virtual Nature

Views generate query results dynamically on the fly from their underlying physical tables (base tables) rather than storing duplicate data on disk.

## Query Simplification

Complex or repetitive operations such as multi-table `JOIN` statements, filtering, and aggregations are encapsulated into simple, reusable queries (e.g., `SELECT * FROM view_name`).

## Security and Data Abstraction

Views enforce row and column-level access control, allowing you to hide sensitive data (e.g., salaries, identification numbers, or contact details) while exposing only the necessary fields to specific users or applications.

## Centralized Business Logic

Centralizing query logic inside a view ensures that all applications and users access data through a single, consistent source of truth.

## Stored Procedures

In relational database management systems, a Stored Procedure is a reusable database object that contains a precompiled set of SQL statements combined with procedural programming logic, such as conditional branching (IF/ELSE), loops, and exception handling. Unlike basic SELECT or INSERT queries sent individually from a client application, stored procedures execute directly on the database server. They allow developers to encapsulate complex, multi-step business logic such as validating seat availability, inserting a flight reservation, and issuing a ticket within a single atomic transaction in a single callable unit. Stored procedures significantly enhance application performance by reducing network traffic, maintaining data integrity through centralized logic, and strengthening security by allowing users to execute predefined business operations without needing direct table modification permissions.

### Procedural Logic

Extends standard SQL by allowing procedural control flow (if-statements, loops, variable declarations, and error handling).

### Performance Efficiency

Precompiled on the server, eliminating the overhead of parsing and optimizing queries each time they run while drastically cutting down client-database network traffic.

### Transaction Control

Essential for executing multi-step business operations safely (e.g., using BEGIN, COMMIT, and ROLLBACK to ensure all steps succeed or none do).

### Security and Encapsulation

Shields underlying table structures and business rules behind executable entry points, preventing direct table manipulation by unauthorized applications.

# Triggers

In relational database management systems, a Trigger is a specialized, event-driven database object that automatically executes a predefined set of SQL statements or procedural code in response to specific Data Manipulation Language (DML) operations, such as INSERT, UPDATE, or DELETE on a designated table or view. Unlike stored procedures, which must be explicitly called by an application or user, triggers execute implicitly "behind the scenes" whenever their associated event occurs. Triggers can be configured to execute before, after, or instead of the triggering action, operating either at the statement level or once for each individual row modified. They provide a critical layer of automation for enforcing complex business rules that exceed static table constraints, maintaining detailed historical audit logs, synchronizing related tables, and safeguarding transactional data integrity directly at the database level.

## Event-Driven Execution

Fired automatically by the database engine when specific DML statements (INSERT, UPDATE, DELETE) occur; cannot be manually executed.

## Timing and Granularity

Configured to run BEFORE, AFTER, or INSTEAD OF an event, executing either once per SQL statement or iteratively FOR EACH ROW.

## Special Pseudo-Records

(NEW and OLD): Provides direct programmatic access to incoming modified data (NEW) and original existing data (OLD) within the trigger body.

## Automated Auditing and Advanced Integrity

Ideal for tracking historical changes (logging user edits and timestamps) and enforcing cross-table validations that declarative CHECK constraints cannot handle.

## Non-relational databases

Relational databases marked a turning point in the technology industry and in the history of how data has been managed throughout human history. The non-relational database model behaves in a way that is a bit closer to the real world.

Relational databases are not the only type of database; they are all subdivided and differ from one another. All of these databases, no matter how different they may be, fall under a single category distinct from relational databases.

## Types of non-relational databases

Relational databases marked a turning point in the technology industry and in the history of how data has been managed throughout human history. The non-relational database model behaves in a way that is a bit closer to the real world.

### Key-Value

They are ideal for quickly storing and retrieving data using a unique key. They handle dictionaries exceptionally well. For more complex queries, it can be difficult to access the values without the key.

They are based on hash algorithms or hash maps (data dictionaries {'key': 'value'})

- DynamoDB
- Cassandra

## Document-based

They are a key-value implementation that differs in how they handle semi-structured information. They are ideal for storing data in JSON and XML formats. They allow you to save the application's current state and restore it later.

They are not as well-suited for sorting and querying data based on different parameters; they do not perform optimally with complex SQL queries.

They may be the most widely used databases outside the SQL standard.

- MongoDB
- Firestore

## Graph-based

These are nodes or entities based on graph theory; they are used for entities that are interconnected by multiple relationships. They are ideal for storing complex relationships.

They are widely used in AI (Artificial Intelligence) to build neural networks that simulate these interdependent, complex relationships.

- New4j
- Titan

## In-memory

They can have various structures, but their advantage lies in their speed, since they reside in memory and data retrieval is almost immediate.

The disadvantage is that they have a certain limit; they must store some data on the computer's storage device and are volatile. If the server restarts, that data will likely have to be retrieved from storage and loaded back into memory, and it takes effort to keep it in sync.

- Memcached
- Redis

## Search-optimized

They can have various structures, and their advantage lies in the fact that they allow for complex queries and searches to be performed easily and quickly.

They serve as large repositories of historical data that can be queried for highly complex business analyses involving BI (Business Intelligence), ML (Machine Learning), business trends, and other types of data.

- BigQuery
- Elasticsearch

## Managed services and data hierarchy

### Firestore

It is a Platform as a Service (PaaS) offering from Google that provides a wide range of options (authentication, databases, storage, hosting, functions, ML kits); it is primarily designed to support web and mobile applications.

### Firestore

It is a managed database service offered by Google that is widely used in the development of web and mobile applications. Firestore is a type of non-relational, document-based database. The Firestore platform includes a command-line interface (CLI) and language connectors such as JavaScript, TypeScript, PHP, and Python. It allows users to connect to and manipulate data.

Firestore offers a free tier where you can use its services up to a certain limit.

# Data Hierarchy in Firestore



Firestore stores documents in a format similar to JSON (JavaScript Object Notation).

Google automatically generates unique IDs for documents in the database.

## Data types in Firestore

### String

Allows you to store any alphanumeric value.

### Number

Allows you to store integer and floating-point values.

### Boolean

Allows you to store any value of type true or false.

### Map

Allows you to add a field within another field (documents within documents).

### Array

Allows you to add an array from a dataset that has no name or identifier.

### Null

Allows you to indicate that the field does not have a defined value.

### Timestamp

Allows you to store any date-type value (year, month, day, and time).

### Geopoint

Allows you to save any integer or floating-point value representing a geographic location using coordinates consisting of latitude and longitude values.

### Reference

Allows you to reference or link to another document; this document can be from the current collection or from another collection.

## Handling data models in non-relational databases

### Top-level collections

Also known as top-level collections, these are the data collections that are immediately available in the project and are intrinsically linked to the database.

### Documents

These are the sets of documents that are created, modified, and deleted within a data collection.

### Subcollections

These are collections of data found within a document. They are not directly related to the database but rather to the document in which they are located.

## Databases in real life

In the real world, there is no such thing as a “one-size-fits-all” database. For quite some time, RDBs were responsible for managing data, but this paradigm broke down as applications became widespread, giving rise to what is known as Big Data and placing greater demands on databases. Although RDBs could handle everything when dealing with large amounts of data in the millions or billions, they began to experience significant issues with consistency and the ability to tailor data usage to specific needs.

## Big data

It is a concept that emerged in response to large amounts of data that could not be handled practically using RDB technologies due to their volume and complexity.

It represents a major shift toward different types of databases that have evolved into a range of solutions designed to handle large volumes of data quickly.

Cassandra is a database developed by Facebook (Meta) that arose from the need to store and retrieve larger volumes of information per second. It has some drawbacks, such as the fact that keys are predefined and it does not allow certain types of JOINS; it is not possible to retrieve all the data at once using a single key, among other limitations that make it less flexible. However, in the specific case of quickly storing and retrieving data, it is an excellent solution.

The term “Big Data” is often confused with “Business Intelligence” because both concepts relate to large volumes of summarized data, analytics, statistics, and business logic.

# THE INTERNET IN 2023 EVERY MINUTE



Created by: eDiscovery Today & LTMG

## Data warehouse

It is the field responsible for storing and manipulating large “repositories” of data. The databases used in this field are unique in that they can store large volumes of data (historical data).

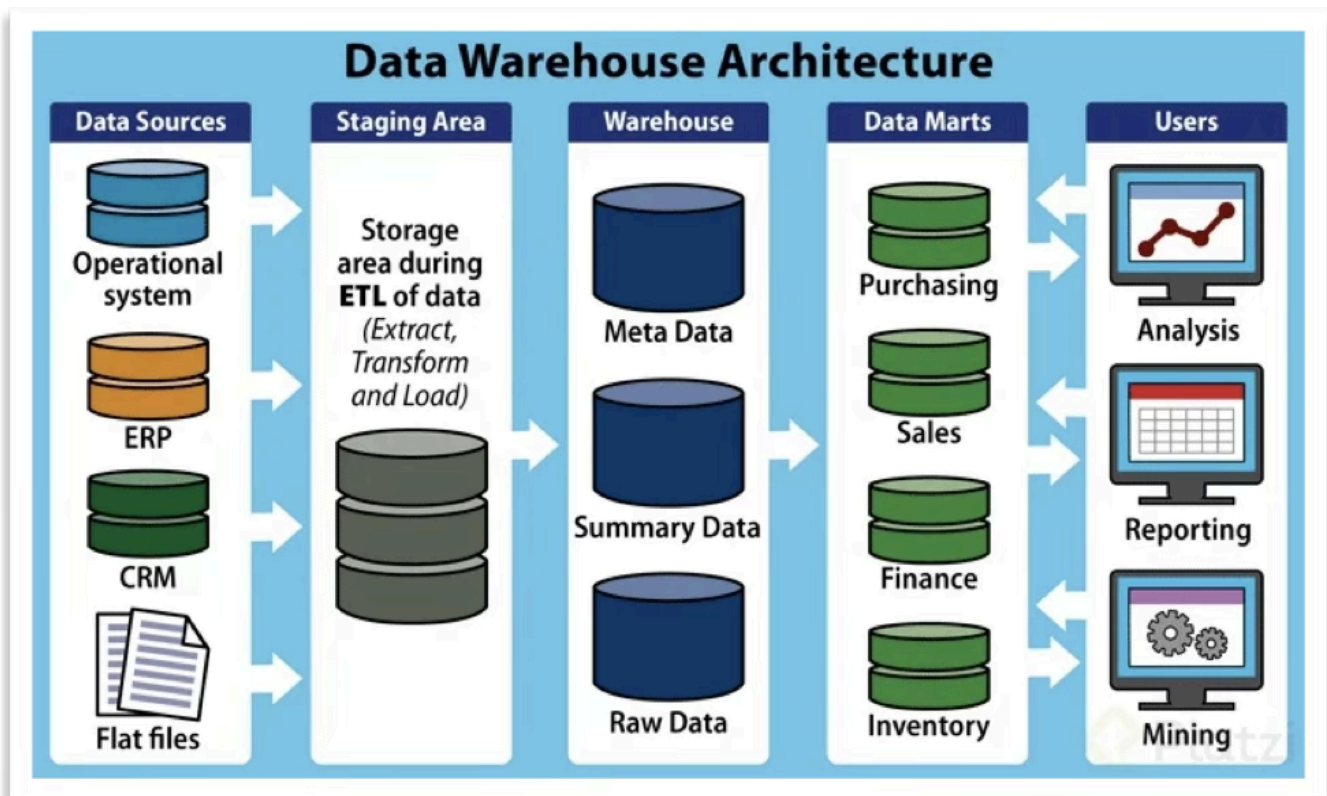
One of the most notable examples is an internal database used by Google called Bigtable; it is unique in that it consists of a single large table, making it a massive data repository. However, it is not very useful for running queries.

Data warehouses are particularly sought after for storing large volumes of data and running queries on this information using analytics and BI.

A data warehouse must support two main activities

1. The ability to store data indefinitely and in large volumes.
2. To be able to extract data and run queries. And to identify patterns that are valuable to the business (BI).

BigQuery is a solution for this type of discipline that allows you to store large volumes of data (it is a columnar database) and enables you not only to run queries on this database but also to aggregate data sources from multiple locations and ultimately generate valuable outputs such as predictions or reports on the company's past performance.



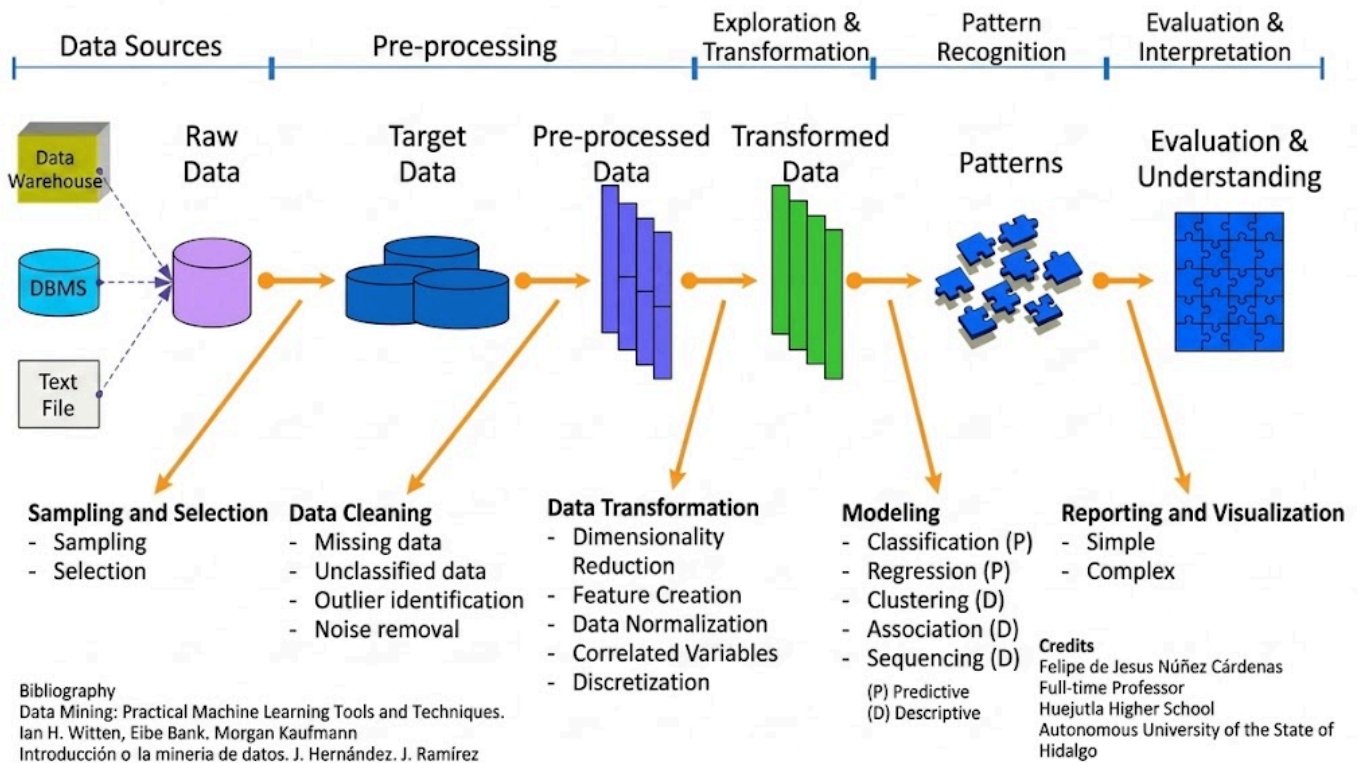
## Data mining

It is a discipline or technique dedicated to mining data (extracting data from wherever it is found) from a legacy file, a current database, or multiple data sources, and making sense of it by applying techniques such as ETL. Many of a company's historical data sources lack a proper system; they are incoherent, lack cohesion, and are unrelated (raw data, unnamed tables, isolated data, unnormalized databases, etc.).

This discipline emerged alongside Data Warehousing, ETL, and Business Intelligence concepts that were closely linked because they addressed new challenges arising from large volumes of data, which historically had not been addressed in the context of data persistence. Initially, everything was handled using RDBs.

All of these new disciplines and/or techniques, together with the extracted data, are applied in Machine Learning or Business Intelligence.

# The Data Mining Process



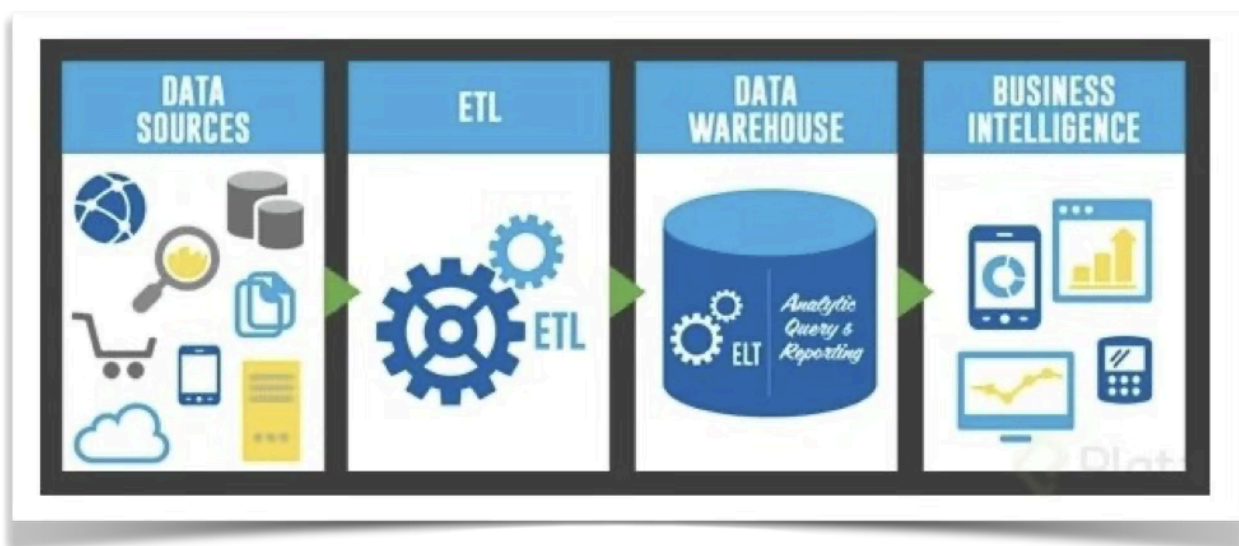
## ETL

ETL (Extract, Transform, and Load) is a widely used tool in the field of data mining. It has two main uses.

1. Data mining is used to extract, transform, and load data into a new database or data warehouse, where the information is organized and structured so it can be used.
2. Just like the first use case, but this time in real time, using a document-based database to track the current status of an application or system, and using a parallel data warehouse database such as BigQuery and connecting both databases via an ETL tool. The frequency is determined by the company's business model. This allows for future analysis using techniques such as data analysis, business intelligence, and machine learning.

ETL is precisely about transforming the data stored in a company's database into valuable information that enables an in-depth analysis of business activity and helps identify where intelligent decisions need to be made based on that data.

This technique is not based on any specific technology, but rather on the theory of data transformation; however, there are some software programs that enable the creation of data pipelines, which are the flow of information from one place to another.



## Business Intelligence

BI (Business Intelligence) is a very special part of data-related disciplines or careers. Its value to business lies in providing information in a timely and clear manner, and in making all the necessary elements available so that the company's leaders can make the right decisions.

Business Intelligence is dedicated to providing stakeholders with the correct data and information for decision-making.

It involves extracting live data from applications or systems, as well as data from a data warehouse, applying data mining and ETL processes to transform it into valuable information. Finally, the BI layer is applied to make sense of all the transformed information, understand relationships, and apply this insight to business logic in the decision-making process.

It employs processes and procedures to ensure that the best possible decisions are made based on the information derived from the data.

It's very useful for

- Create reports.
- Create data tables.
- Dashboards.
- Create charts.
- Make decisions.



# Machine Learning

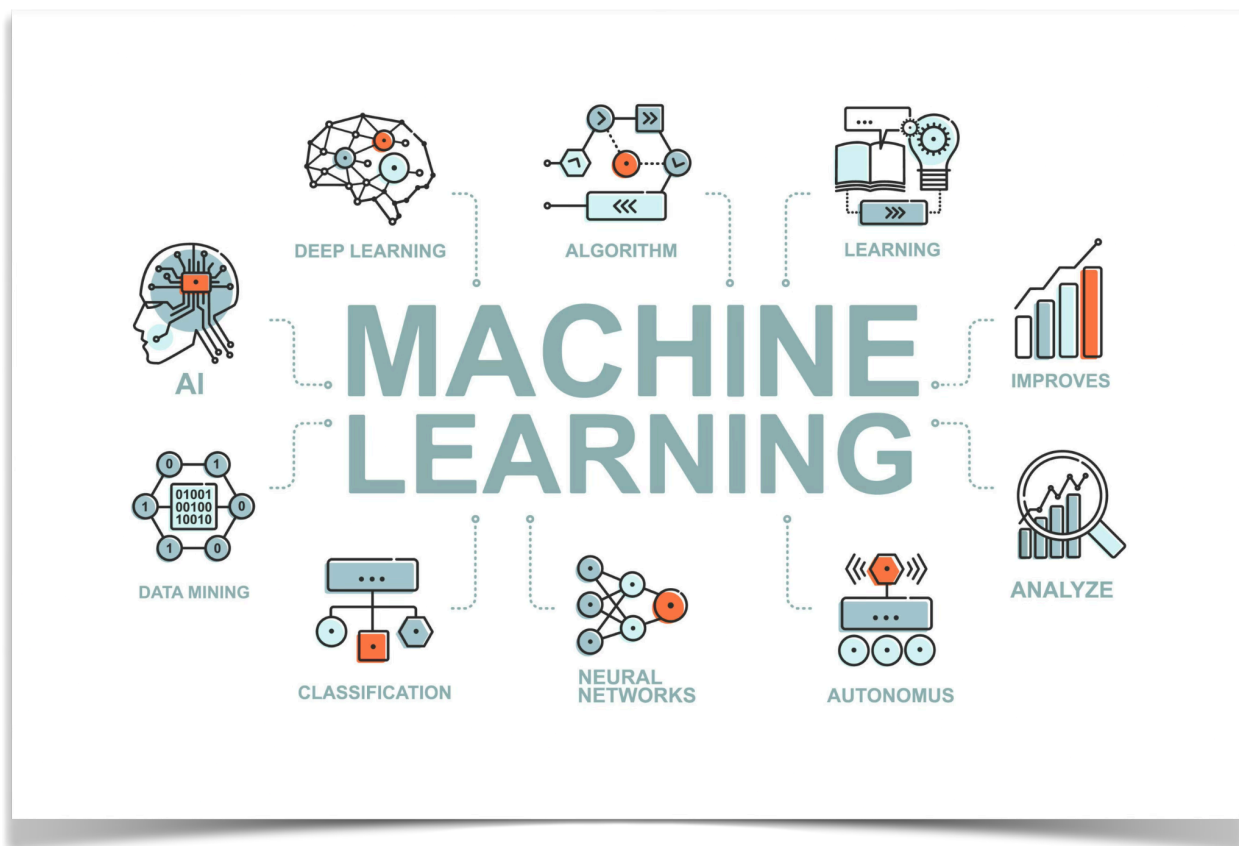
ML (Machine Learning) has a very broad meaning and depends on the context.

It is a set of techniques involving AI (Artificial Intelligence) and pattern recognition in various fields.

ML has a wide range of applications in data-related disciplines; it is primarily used in data processing to identify patterns of interest. It is also viewed as the next step beyond BI.

ML enables the creation of models that identify hidden patterns; in other words, ML aims to uncover relationships or patterns that are not immediately obvious. Most ML models are based on statistics.

BigQuery is a database; in addition to being used for data warehousing, it also employs ML for data processing.



ML has two core algorithms or models with specific use cases

## Classification

It involves large volumes of data. Using natural language processing techniques as input, it takes a set of data that may come from various sources, processes it, and ultimately identifies the patterns found.

## Prediction

It involves decision-making, using a large volume of historical data as input, training a model, processing the data, and, as output, generating a trained model with a specific accuracy rate. Data is fed into this trained model for processing, and it identifies the patterns that influence and shape the relationships within the data.

ML models are based on linear algebra, linear regression, tensors, and calculus.

ML and today's tools do not focus primarily on generating new models (although that is possible) but rather on knowing how and where to apply them.

The greatest need for ML in the industry today is not for brilliant mathematicians but for experts in information and data processing.

## Data science

Data Science is a discipline that encompasses the techniques of Big Data, Data Warehousing, Data Mining, ETL, Business Intelligence, and Machine Learning.

People who pursue this discipline typically have a background in statistics, mathematics, and other sciences. However, it is now complemented by other disciplines such as Data Engineering, which deals with data transformation.

The data scientist also serves in a managerial role, as they are the ones who put knowledge and projects into practice.

